

TUTORIAL AWS · DESPLIEGUE CLOUD

DEL CLOUD A TU PRIMERA WEB EN AWS

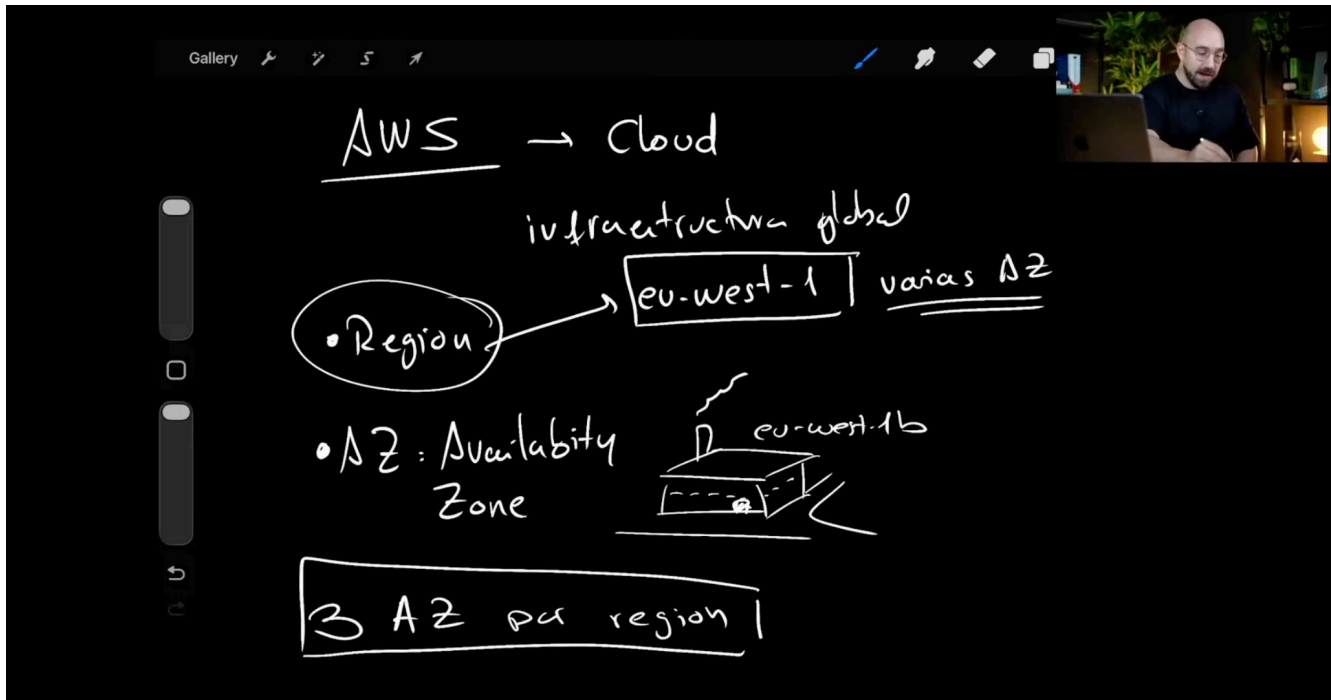
Despliegue completo de una web estática con Amazon S3 y
CloudFront

Betta Tech

Qué es el cloud

Cloud computing, o computación en la nube, consiste en acceder a recursos informáticos bajo demanda mediante servicios de un proveedor. En lugar de comprar y mantener servidores físicos, puedes contratar capacidad de cálculo, almacenamiento o bases de datos cuando las necesitas. Habitualmente el precio depende del uso, aunque cada servicio puede tener distintas modalidades de contratación. [14]

La nube sigue siendo infraestructura física: ordenadores, discos, redes y centros de datos. Lo que cambia es cómo accedes a ella y quién se ocupa de cada capa. Desde una consola o una API solicitas recursos; el proveedor gestiona la infraestructura que hace posible ofrecerlos.



De comprar capacidad a solicitarla

Imagina que quieres publicar Quiz Arena desde un servidor propio. Tendrías que conseguir una máquina, conectarla, mantenerla y prever cuánto tráfico soportará. En la nube puedes crear los recursos necesarios sin comprar ese hardware. Esto facilita experimentar, pero no elimina la necesidad de diseñar, configurar y supervisar la aplicación.

La **escalabilidad** describe la capacidad de atender más carga. La **elasticidad** añade la posibilidad de aumentar y reducir recursos conforme cambia la demanda. El cloud facilita ambas, pero el ajuste automático depende del servicio y de su configuración: contratar una máquina virtual no hace que cualquier aplicación escale por sí sola. [14]

El pago por uso necesita contexto

Consumir recursos bajo demanda permite adaptar el gasto, pero “pago por uso” no significa pagar solo cuando hay visitantes. Un servidor encendido o unos archivos almacenados pueden generar coste aunque nadie abra la web. Más adelante veremos qué se cobra en nuestra arquitectura y dónde revisar la factura.

También hay que distinguir cloud de acceso público. Un recurso alojado en la nube puede ser privado. En este tutorial, los archivos vivirán en S3 con acceso anónimo bloqueado y la web se entregará por CloudFront. Su ubicación y sus permisos son decisiones diferentes.

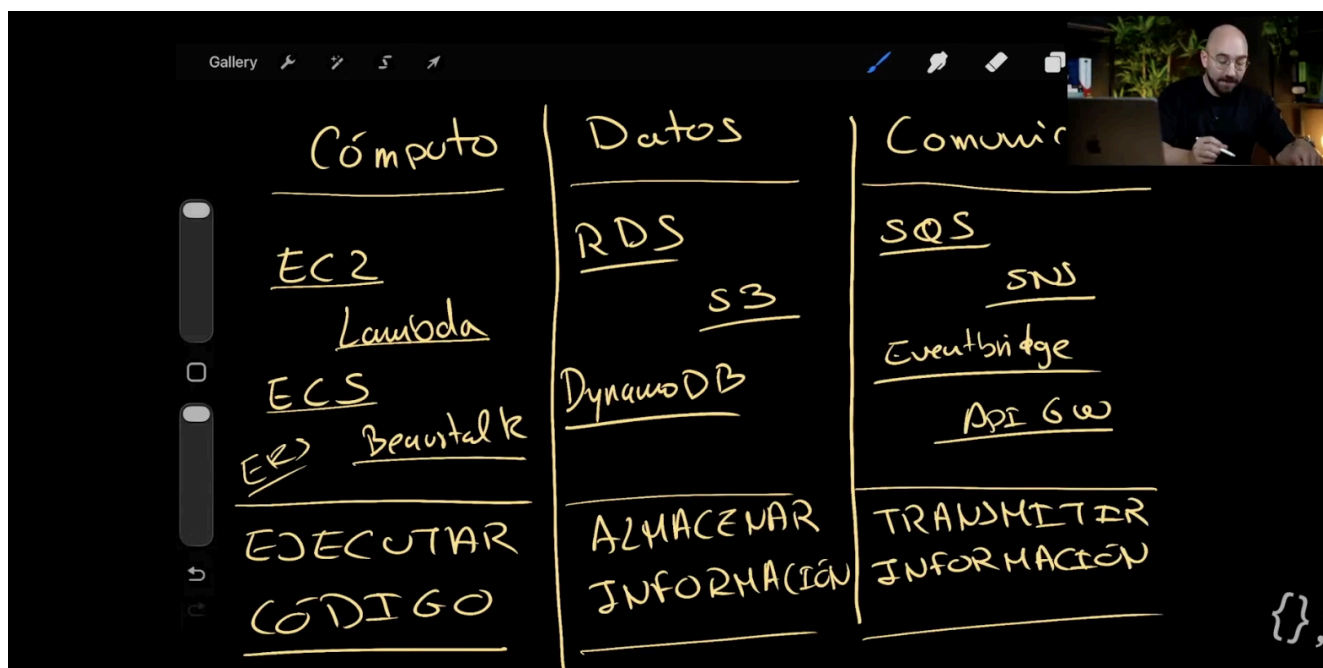
Qué es AWS y cómo se utiliza

AWS significa Amazon Web Services. Es la plataforma de servicios cloud de Amazon: ofrece piezas para ejecutar aplicaciones, guardar datos, conectar sistemas y gestionar acceso, entre otras funciones. No es un único servidor ni una herramienta que resuelva todos los problemas del mismo modo. [15]

Servicios y recursos

Un **servicio** es una capacidad que ofrece AWS. Un **recurso** es una instancia concreta que creas o utilizas dentro de ella. Amazon S3 es el servicio de almacenamiento de objetos; el bucket de Quiz Arena es uno de nuestros recursos. Amazon EC2 es un servicio de cómputo; una máquina virtual concreta es una instancia EC2.

Esta distinción permite leer la consola con más claridad. Primero eliges el servicio que resuelve tu necesidad; después creas recursos y configuras cómo se relacionan. Para entregar un archivo no siempre necesitas la misma pieza que para ejecutar un proceso o consultar una base de datos.



Consola y automatización

AWS Management Console es la interfaz web para administrar recursos. **AWS CLI** permite realizar operaciones desde el terminal y los **SDK** permiten integrarlas en código. Son distintas formas de trabajar con los servicios: la arquitectura no cambia porque sustituyas una subida manual por un comando.

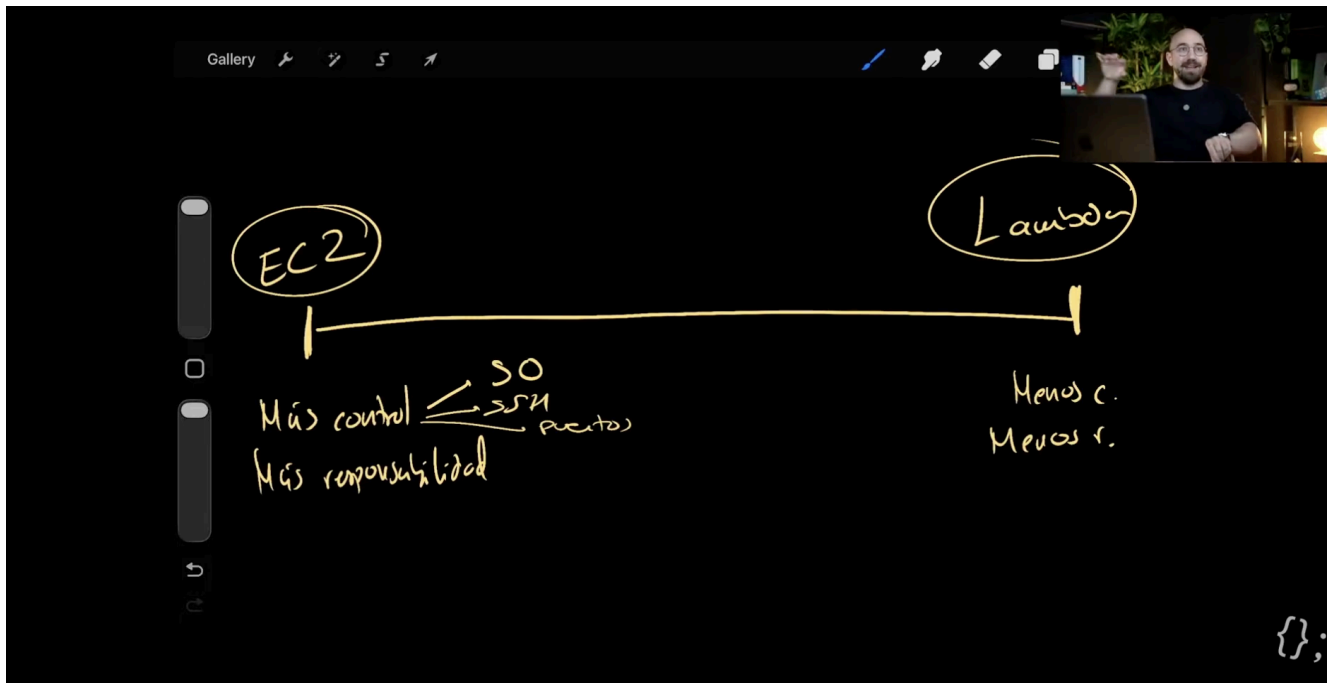
La cuenta de AWS agrupa el uso y la administración de los recursos. Los permisos determinan qué puede hacer cada identidad. Por eso debes separar dos preguntas: “¿en qué cuenta estoy trabajando?” y “¿qué acciones tengo autorizadas?”. Encontrar un servicio en la consola no significa tener permiso para modificarlo.

Cuánto control quieres asumir

Con una máquina virtual administras el sistema operativo y el software que ejecuta. Con un servicio más gestionado delegas parte de esa operación y te concentras en su configuración y en tus datos. Cuanto más control necesitas sobre una capa, más trabajo suele quedar de tu lado.

Las siglas **IaaS**, **PaaS** y **SaaS** ayudan a situar ese espectro. IaaS proporciona infraestructura; PaaS añade una plataforma para desplegar aplicaciones; SaaS entrega una aplicación lista para usar. Son categorías orientativas: al elegir un servicio concreto, lo útil es comprobar qué gestiona el proveedor y qué sigues gestionando tú. [14]

En Quiz Arena evitaremos operar una máquina solo para servir archivos. S3 almacenará el contenido y CloudFront lo entregará. Esa elección reduce tareas de administración, pero los permisos y los archivos que publicamos seguirán siendo responsabilidad nuestra.



Regiones zonas y ubicaciones de borde

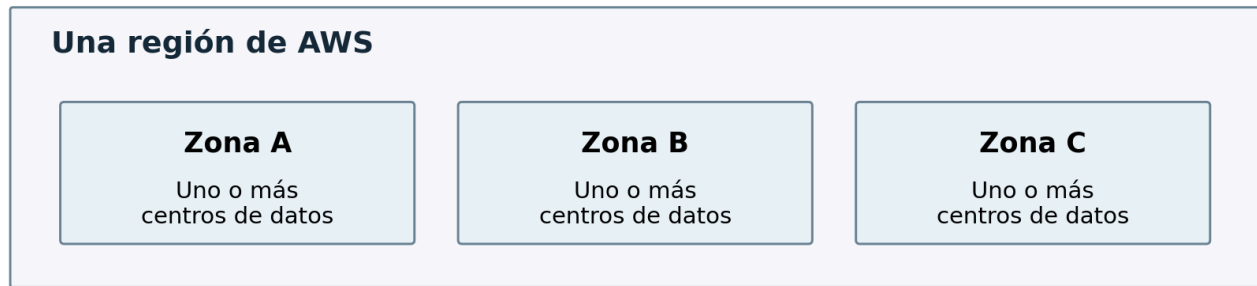
La nube tiene geografía. AWS organiza su infraestructura en **regiones**, y cada región contiene varias **Availability Zones** o zonas de disponibilidad. Una AZ puede incluir uno o más centros de datos: no es simplemente otro nombre para un único edificio. [1]

Una región sitúa tus recursos

La región es el área geográfica donde creas muchos de tus recursos. Al elegirla, considera la proximidad a los usuarios, los requisitos de ubicación de datos, los servicios disponibles y el precio. En la consola, comprueba el selector regional: estar mirando otra región puede explicar que no encuentres un recurso regional que acabas de crear.

Las zonas permiten separar fallos

Las AZ de una región están aisladas físicamente y conectadas mediante redes de baja latencia. La separación permite diseñar aplicaciones que continúen funcionando si una zona tiene problemas. Sin embargo, disponer de varias zonas no significa que tu aplicación las esté utilizando: depende del servicio y del despliegue. [1]



Una región contiene varias AZ. Esquema conceptual con tres zonas.

La analogía del meteorito lo hace visible: si toda la aplicación depende del datacenter que queda destruido, pierde su punto de apoyo. Repartir componentes entre zonas reduce ese riesgo. Para un incidente que afecte a una región completa haría falta una estrategia entre regiones; no basta con decir que la aplicación está “en AWS”.

CloudFront acerca contenido a los visitantes

Las **ubicaciones de borde** son puntos de la red que CloudFront (Frente de Nube, llamado en **broma** durante las clases) utiliza para entregar contenido cerca del usuario. No son las AZ que eliges para una máquina virtual. El bucket puede estar en una región y CloudFront servir copias en caché desde su red distribuida. Eso no convierte al bucket en un almacenamiento replicado automáticamente entre regiones. [8]

Con este mapa ya podemos entender el recorrido práctico: elegiremos dónde guardar los archivos, quién puede leerlos y cómo llegan al navegador. Empezaremos por el problema que resuelve cada servicio y después trasladaremos esas decisiones a la consola.

Cómo publicar una web estática en AWS

Un tutorial de S3 y CloudFront

Publicar una web no siempre requiere administrar un servidor. Si la aplicación que queremos entregar al navegador está formada por HTML, CSS y JavaScript, podemos guardar esos archivos en **Amazon S3** y distribuirlos mediante **CloudFront**. Ese es el recorrido de este tutorial: entender la arquitectura y después construirla, manteniendo privado el almacenamiento de origen.

Al terminar, Quiz Arena tendrá una URL HTTPS desde la que cualquier persona podrá cargar la web. El bucket de S3 seguirá bloqueando el acceso anónimo directo. Esta distinción es esencial: protegemos el origen, pero no estamos añadiendo un sistema de inicio de sesión a la aplicación.

Primero entender la herramienta

Aprender AWS como una secuencia de clics resulta frágil. Una pantalla cambia de sitio y el procedimiento deja de servir. Es más útil empezar por las decisiones: qué necesita ejecutarse, qué necesita almacenarse y cómo llegará cada petición a su destino. La consola será una forma de expresar esas decisiones; más adelante podremos hacer lo mismo con comandos o infraestructura como código.

Pensemos en un VPS. Alquilar esa máquina nos permite instalar un servidor web y publicar los archivos, pero también nos obliga a ocuparnos del sistema operativo, sus parches y la disponibilidad del proceso. AWS puede ofrecernos ese modelo mediante EC2, aunque su catálogo va mucho más allá: almacenamiento, bases de datos, identidad y redes que podemos combinar sin administrarlo todo dentro de una sola máquina.

La comodidad también forma parte del coste

La comparación con Vercel ayuda a situar el nivel de abstracción. Una plataforma orientada al despliegue web resuelve por nosotros muchas decisiones y reduce el trabajo necesario para publicar. Al usar AWS directamente, elegimos más piezas y asumimos su configuración. Ganamos control, pero invertimos tiempo en comprender cómo encajan.

Por eso conviene comparar el coste total, no solo la factura de infraestructura. Para un equipo, una plataforma más abstracta puede compensar por las horas de operación que ahorra. Para otro, configurar servicios de AWS directamente puede encajar mejor. Ninguna de las dos opciones garantiza por sí sola ser la más barata.

Qué vamos a desplegar

En esta primera etapa, Quiz Arena es un conjunto de archivos web. El navegador interpreta el HTML, aplica el CSS y ejecuta el JavaScript. Aunque la interfaz permita interactuar, el alojamiento sigue siendo estático: S3 entrega archivos, no ejecuta la lógica de un backend. Si después necesitamos una API, añadiremos un servicio que sí ejecute esa lógica.

La arquitectura antes de la consola

Para orientarnos en AWS podemos dividir las necesidades en tres bloques: **Compute** para ejecutar código, **Data** para guardar información y **Communication** para conectar componentes y usuarios. Es un mapa didáctico, no una clasificación exhaustiva. En Quiz Arena, el navegador ejecuta el JavaScript, S3 guarda los archivos y CloudFront se encarga de entregarlos por Internet.

La petición empieza en el navegador y llega a CloudFront. Si este dispone de una copia válida en caché, responde con ella. Si necesita consultar S3, lo hace con una solicitud firmada mediante **Origin Access Control (OAC)**. La política del bucket permite esa lectura a la distribución autorizada. [7, 8]



Figura 1. La web se publica por CloudFront y el origen S3 permanece privado.

Por qué guardar archivos en S3

S3 es almacenamiento de objetos. Un bucket contiene objetos y cada objeto tiene una clave: index.html o assets/app.js, por ejemplo. Aunque la consola muestre carpetas, assets/ es un prefijo de la clave; no estamos trabajando con el disco de una máquina que debamos mantener. [3]

Usar EC2 para entregar estos mismos archivos sería posible, pero añadiría un servidor web que instalar, actualizar y supervisar. Para esta aplicación, S3 evita ese trabajo. Pagamos por el uso del almacenamiento y

las operaciones, además de la entrega de contenido que corresponda. La ventaja es un encaje más simple con el problema, no una promesa de gratuidad.

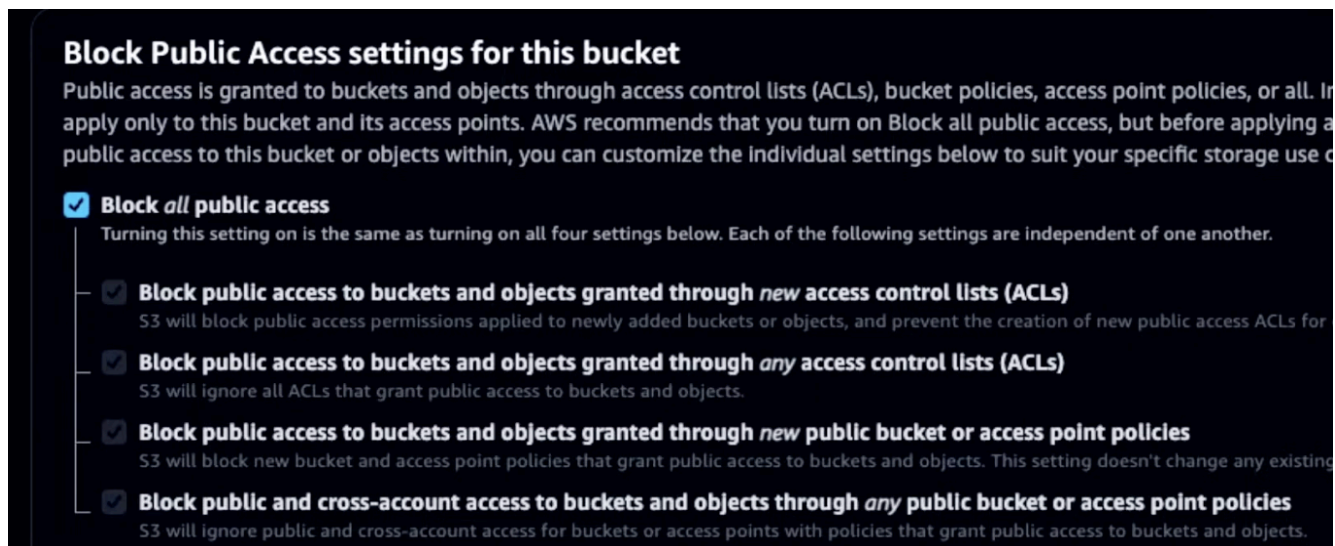
Aplicar la resiliencia al almacenamiento

S3 Standard almacena los datos en varias AZ de la región; las clases One Zone tienen otra característica de resiliencia. Elegir una clase de almacenamiento forma parte del diseño. Esa redundancia no sustituye una estrategia de recuperación frente a borrados accidentales. [3]

La seguridad que sigue siendo tuya

Delegar la operación de un servicio no significa delegar todas sus decisiones de seguridad. AWS lo explica mediante el **modelo de responsabilidad compartida**: AWS protege la infraestructura de la nube y el cliente configura de forma segura lo que utiliza en ella. El reparto cambia según el servicio contratado. [2]

En EC2, por ejemplo, seguirías administrando el sistema operativo invitado y la aplicación. En S3 no aplicas parches al servidor de almacenamiento, pero sí decides quién puede leer o subir objetos. Para Quiz Arena, nuestras decisiones principales serán mantener el bucket privado y conceder a CloudFront únicamente la lectura necesaria.



Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. It applies only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying a public access setting to this bucket or objects within, you can customize the individual settings below to suit your specific storage use case.

- ☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for buckets and objects.
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies.
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Un bucket privado no convierte la web en privada

Cuando digamos que S3 solo admite a CloudFront, nos referimos a la vía de entrega pública de esta arquitectura. Las identidades de administración o despliegue pueden seguir accediendo si sus permisos lo permiten. OAC tampoco autentica a quienes visitan la web: autoriza el salto de CloudFront al origen. [7]

Nota importante de seguridad:

Esto tiene una consecuencia práctica: cualquier archivo que CloudFront entregue al navegador debe considerarse publicable. No incluyas secretos, claves de AWS o contraseñas en el JavaScript. El usuario puede inspeccionar lo que recibe, aunque el bucket sea privado.

Prepara los archivos antes de entrar en AWS

Localiza la carpeta que contiene la versión publicable de la aplicación. En los ejemplos la llamaremos **dist/**. Si tu proyecto no tiene un proceso de compilación, utiliza la carpeta donde estén el HTML y sus recursos. Lo importante es que `index.html` se encuentre en la raíz de lo que vas a subir y que sus referencias a CSS, JavaScript e imágenes coincidan con las rutas reales.

```
None
dist/
  index.html
  styles.css
  app.js
  assets/
    logo.png
```

Revisa la web en local antes de desplegarla. Si falta un archivo o una ruta ya está mal, subirla a AWS no corregirá el problema. También conviene separar el resultado publicable del código auxiliar, la configuración de desarrollo y cualquier fichero que no deba salir del equipo.

Para la primera subida basta la consola y una identidad con permisos adecuados. Para repetirla desde el terminal necesitaremos AWS CLI instalada, credenciales válidas y permisos para listar el bucket y subir objetos. La identidad que despliega y el permiso con el que CloudFront lee son dos cosas distintas.

Paso 1 Crear el origen privado en S3

Abre S3 y crea un bucket de tipo **General purpose** en la región elegida. El bucket será el origen de los archivos; todavía no necesitamos hacerlo accesible desde Internet. Antes de crearlo, presta atención al nombre y a las opciones de acceso público.

El nombre no determina dónde se replican los datos

S3 ofrece un namespace global compartido y un namespace regional de cuenta. En el global, el nombre debe ser único entre cuentas y regiones de una misma partición de AWS. En el regional de cuenta, se utiliza un sufijo con el identificador de cuenta, el código de región y `-an`, reservando esos nombres a esa cuenta. [4]

```
None
Global compartido: quiz-arena-web-<sufijo-unico>
Regional de cuenta: quiz-arena-<cuenta>-<region>-an
```

Ambas opciones crean un bucket en una región. Aquí “global” describe el espacio de nombres, no significa que los archivos se repliquen por todo el planeta. Copia el nombre completo que tenga el bucket: lo utilizarás en los comandos y en la política de permisos.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☐ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☒ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name
For general purpose buckets in the account Regional namespace, the bucket name is composed of a bucket name prefix and a suffix. AWS adds the suffix to the bucket name prefix that you provide. This suffix is the account Regional namespace. The bucket name can't be edited after the bucket is created.

Bucket name prefix
kdhasdh

Account Regional namespace suffix
-635506416867-eu-west-1-an

The bucket name prefix and the account Regional suffix combined must be between 3 and 63 characters long. The specified bucket name prefix must be unique within your account Regional namespace and follow the [bucket naming rules](#).

Full bucket name
kdhasdh-635506416867-eu-west-1-an

Mantén bloqueado el acceso público

Deja activadas las cuatro opciones de **Block all public access** y conserva las ACL deshabilitadas con **Bucket owner enforced**. No hace falta abrir el bucket para publicar la web por CloudFront. La autorización específica entre ambos servicios se configurará después. [7]

Para esta arquitectura tampoco utilizaremos el endpoint de static website hosting de S3: OAC necesita el origen S3 normal. Es una diferencia importante respecto de otros tutoriales que publican una web directamente desde S3. [7]

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying a public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use d

☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

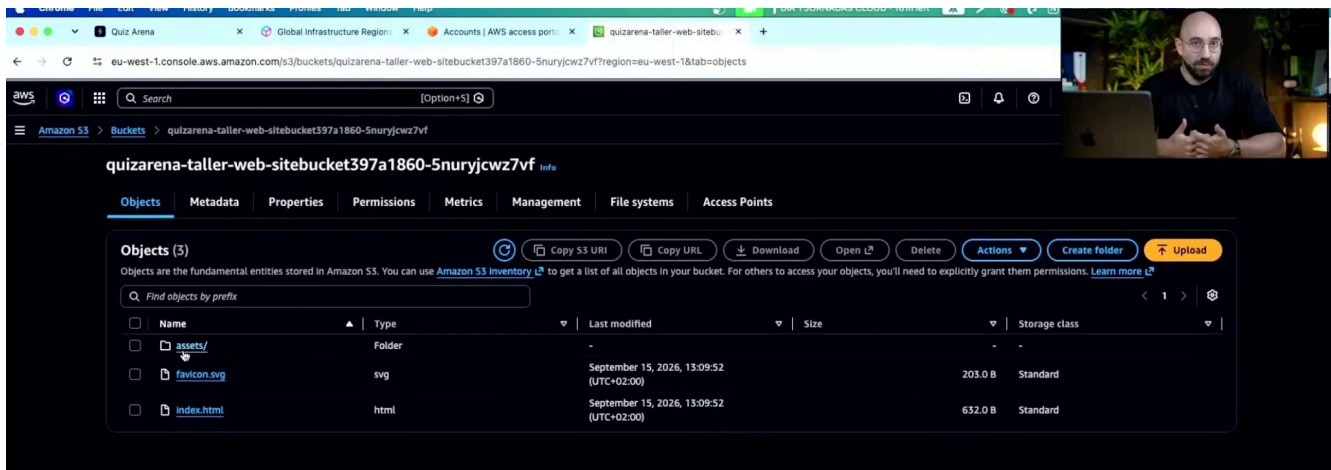
- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Sube el contenido de la carpeta publicable

Entra en el bucket y carga index.html, los estilos, el JavaScript y las imágenes. Sube el contenido de dist/, de forma que la clave del documento de entrada sea index.html y no dist/index.html. Conserva las rutas relativas de los recursos.

Comprueba los nombres exactos, incluidas mayúsculas y minúsculas, y los metadatos Content-Type de los archivos. Un recurso solicitado como app.js no se encontrará si lo has subido con otra clave. Estos detalles suelen explicar más fallos iniciales que la propia infraestructura.

En este punto, intentar leer un objeto mediante una URL anónima de S3 debe dar acceso denegado. Es el estado que buscamos. El siguiente paso será habilitar una puerta de entrada pública en CloudFront, sin cambiar esa decisión sobre el bucket.



Paso 2 Conectar CloudFront con el bucket

Crea una distribución de CloudFront y selecciona el bucket como origen S3. En Origins debe aparecer el destino del que se recuperarán los archivos. Usa el endpoint normal del bucket, no el endpoint web de S3.

Configura index.html como **Default root object**, sin barra inicial. Así, cuando alguien visite la raíz del dominio, CloudFront sabrá qué documento solicitar. Para esta web basta permitir GET y HEAD, y puedes configurar la redirección de HTTP a HTTPS. [6]

Autorizar la lectura mediante OAC

En el origen, crea o selecciona un OAC de tipo S3 y utiliza **Sign requests**, la opción recomendada. Después aplica al bucket la política de lectura que corresponda a esa distribución. Ambas configuraciones son necesarias: firmar una petición no basta si S3 no autoriza su lectura. [7]

Este ejemplo muestra la estructura. Sustituye MI_BUCKET, ID_CUENTA e ID_DISTRIBUCION por los valores reales; también puedes revisar y utilizar la política que proporcione la consola.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "cloudfront.amazonaws.com"},
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::MI_BUCKET/*",
    "Condition": {"StringEquals": {
```

```

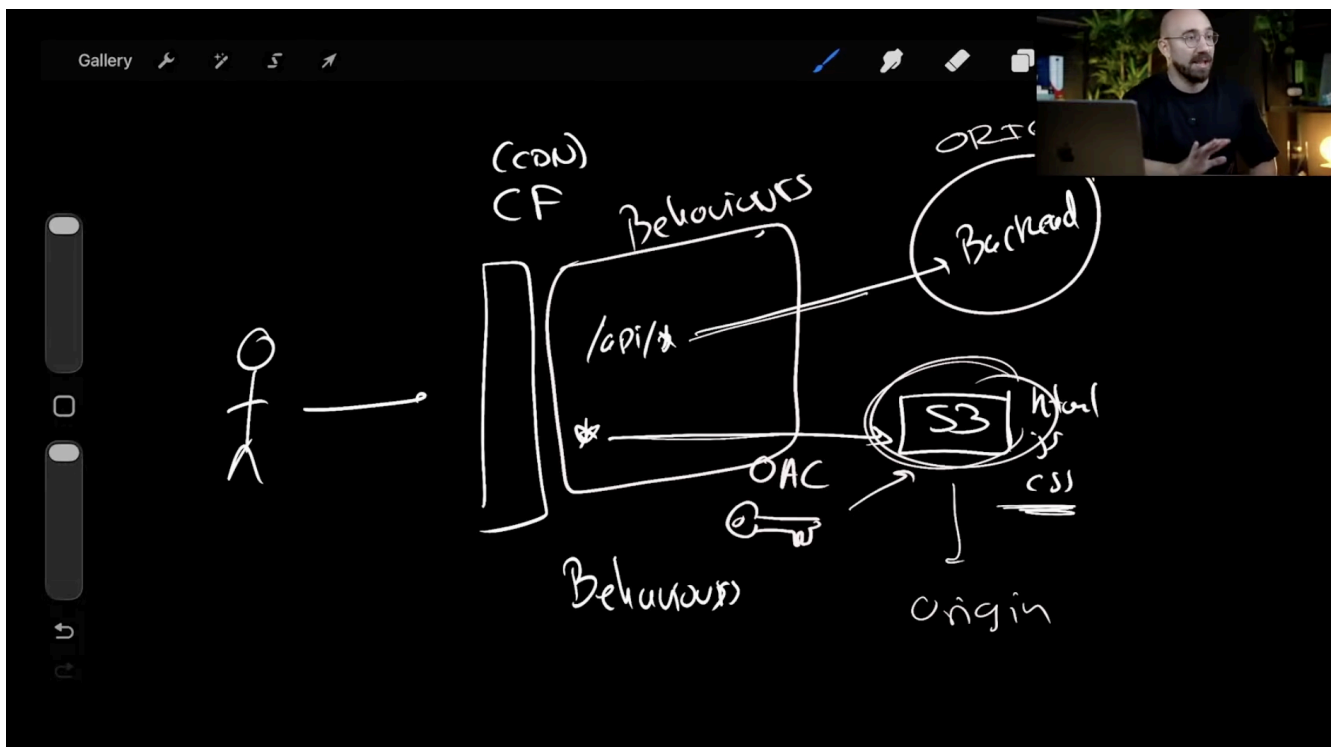
    "AWS:SourceArn":
      "arn:aws:cloudfront::ID_CUENTA:distribution/ID_DISTRIBUCION"
  }}
}]
}

```

La acción permite leer objetos y la condición limita el permiso a una distribución concreta. No concede escritura. Si los objetos usan SSE-KMS, hay que conceder también los permisos adecuados sobre la clave KMS. Guarda los cambios y espera a que termine el despliegue de la distribución. [7]

Entender Origins y Behaviors

Origins responde a “¿de dónde sale el contenido?”. **Behaviors** responde a “¿qué reglas se aplican a esta ruta?”. Default (*) cubre las peticiones que no coinciden con un comportamiento anterior. Más adelante, /api/* podría apuntar a otro origen que implemente una API. Esa regla no crea un backend ni elimina automáticamente el prefijo /api; solo selecciona el destino y sus ajustes. El orden de coincidencia importa. [8]



Paso 3 Comprobar cómo se entrega la web

Abre la URL HTTPS de la distribución. Deberías ver Quiz Arena y poder cargar sus recursos. Ahora conviene comprobar dos cosas diferentes: que la web funciona y que sigue siendo imposible saltarse CloudFront mediante una petición anónima al objeto de S3.

Observar la caché en DevTools

Abre las herramientas del navegador y entra en Network. Selecciona una petición real a index.html o a un recurso JavaScript y busca **X-Cache** entre las cabeceras de respuesta. **Miss from cloudfront** indica que esa petición no se ha resuelto como un acierto de caché. **Hit from cloudfront** indica un acierto. Un Miss inicial es normal.

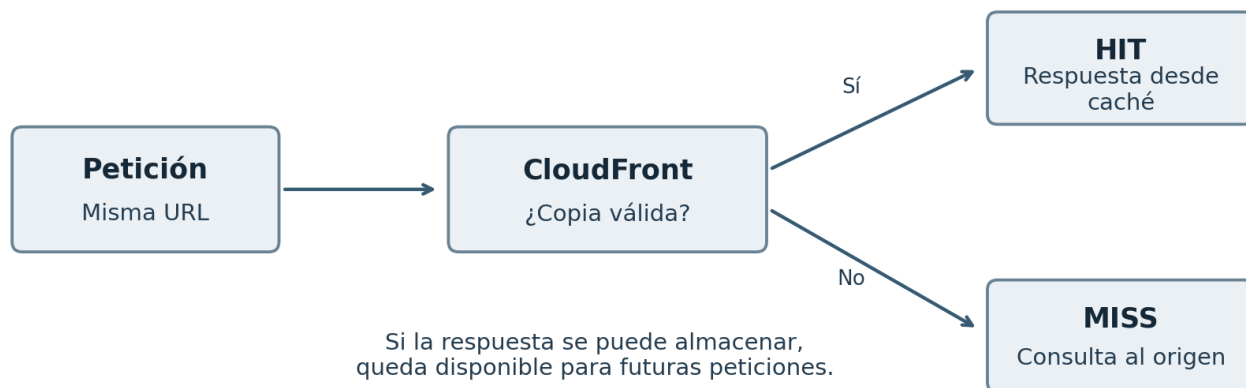


Figura 2. La caché puede evitar una nueva consulta al origen.

Repite la petición a la misma URL. Si el contenido es almacenable y sigue siendo válido, una siguiente respuesta puede ser un Hit. No está garantizado en cualquier recarga: influyen las reglas de caché, su expiración y la ubicación que atiende la petición. Desactivar la caché del navegador ayuda a observar tráfico real, pero no borra la de CloudFront.

CloudFront acerca copias del contenido a los usuarios y puede reducir lecturas a S3. Eso suele mejorar la latencia, pero no elimina los costes de entrega. Sus políticas definen qué forma parte de la clave de caché y cuánto tiempo se puede conservar una respuesta. [8, 11]

Comprobar la protección del origen

Desde una sesión sin credenciales, intenta acceder a un objeto existente por su URL directa de S3. El acceso debe denegarse, mientras la URL de CloudFront funciona. Ver un Hit no demuestra por sí solo que OAC esté configurado correctamente: la prueba de acceso directo comprueba una propiedad distinta.

Si CloudFront devuelve un 403, revisa que el objeto exista, el origen sea el bucket correcto y el ARN de la política corresponda a esa distribución. Si la raíz falla pero /index.html funciona, revisa Default root object. Si solo faltan estilos o imágenes, empieza por sus rutas y las claves subidas.

Paso 4 Publicar cambios de forma repetible

La subida manual sirve para conocer S3, pero no es cómoda para cada nueva versión. Con AWS CLI puedes sincronizar la carpeta publicable con el bucket. Primero revisa las operaciones previstas y, después, realiza la copia. Sustituye MI_BUCKET por el nombre completo del bucket. [5]

Shell

```
aws s3 sync ./dist/ s3://MI_BUCKET/ --dryrun  
aws s3 sync ./dist/ s3://MI_BUCKET/
```

sync copia archivos nuevos o modificados según sus criterios de comparación. No borra automáticamente los objetos que sobran en el destino; **--delete** permite hacerlo y debe usarse entendiéndose qué eliminará. No añadas **--acl public-read**: el despliegue debe conservar el modelo de acceso privado. [5]

Por qué una subida no siempre se ve al instante

Imagina que cambias `index.html` y lo vuelves a subir. S3 ya tiene la nueva versión, pero CloudFront puede seguir conservando una copia válida de la anterior. Además, el navegador puede tener su propia copia. Son capas diferentes; ejecutar `sync` no las vacía.

Hay tres formas habituales de gestionar el cambio: esperar a la expiración, invalidar rutas o utilizar nombres versionados. Para comprobar una modificación del HTML durante la práctica, puedes crear una invalidación de sus rutas de entrada. [9]

Shell

```
aws cloudfront create-invalidation \  
  --distribution-id ID_DISTRIBUCION \  
  --paths "/index.html" "/"
```

La invalidación retira contenido de las cachés de CloudFront, pero no borra el objeto de S3 ni la caché del navegador. Invalidar `"/"` resulta sencillo para una práctica; en un flujo habitual conviene limitar las rutas y considerar el coste aplicable. [9, 11]

Para CSS y JavaScript es común generar nombres con un hash, como `app.a1b2c3.js`. Una nueva versión recibe otra URL y el HTML apunta a ella. Así no dependemos de que desaparezca una copia antigua con el mismo nombre. El HTML, que conecta esas versiones, sigue necesitando una estrategia de caché adecuada. [9]

Un ciclo de despliegue que puedas explicar

Prepara y comprueba los archivos, revisa la sincronización con `--dryrun`, sube el resultado y actualiza las cachés que corresponda. Después abre la aplicación y revisa una petición real. Lo importante es poder distinguir si un fallo procede del archivo publicado, de sus permisos o de una copia anterior todavía almacenada.

Un dominio propio y una factura comprensible

Cuando la web funciona con la URL de CloudFront, añadir un dominio propio es un paso separado. **Route 53** puede gestionar el DNS y apuntar el nombre a la distribución mediante un registro `alias`. En CloudFront también debes registrar ese nombre como dominio alternativo y asociar un certificado que lo cubra. [10]

Si usas un certificado de **AWS Certificate Manager** para el HTTPS entre los visitantes y CloudFront, solicítalo o impórtalo en us-east-1. Esa ubicación del certificado no obliga a que el bucket de la aplicación esté en la misma región. [13]

Mirar el coste desde el primer despliegue

Abre **Billing and Cost Management** y revisa qué servicios generan consumo. En este proyecto, piensa en el almacenamiento y las solicitudes de S3, la entrega y el plan de CloudFront y, si los añades, el dominio y el DNS. El tráfico servido desde caché puede ahorrar consultas al origen, pero la entrega sigue teniendo su propio modelo comercial. [11]

No hace falta memorizar precios para perder el miedo a la factura. Hace falta identificar los recursos creados, entender qué uso se cobra y revisar las condiciones que se aplican a la cuenta. Configurar un presupuesto en **AWS Budgets** permite recibir avisos sobre gasto real o previsto; un aviso no es un límite duro y los datos no son instantáneos. [12]

Qué deberías poder explicar al terminar

S3 guarda los archivos de Quiz Arena sin exponerlos de forma anónima. CloudFront ofrece la URL pública, entrega las respuestas y puede reutilizar contenido en caché. OAC firma el acceso al origen y la política del bucket autoriza a la distribución. El navegador recibe los archivos y ejecuta el JavaScript.

Si mañana necesitas una API, ya tienes un lugar conceptual para situarla: un servicio de Compute que implemente la lógica y un behavior que dirija las peticiones adecuadas hacia su origen. Crear la ruta no implementará esa lógica. Del mismo modo, si quieres que solo entren usuarios identificados, tendrás que diseñar autenticación; OAC no resuelve ese requisito.

Comprobación final

- La web y todos sus recursos cargan desde la URL HTTPS de CloudFront.
- El bucket conserva Block all public access y rechaza lecturas anónimas directas.
- El origen usa OAC y la política referencia la distribución correcta.
- Puedes publicar un cambio y explicar cuándo lo verá el navegador.
- Sabes dónde revisar el consumo y qué recursos has creado.

Ese es el objetivo de esta primera clase: que el despliegue no sea una sucesión de pantallas, sino una arquitectura que puedes razonar, comprobar y volver a construir.

Referencias para seguir aprendiendo

La documentación oficial amplía los conceptos y ajustes del tutorial. Los números entre corchetes remiten a estos enlaces. Las pantallas de la consola y las condiciones comerciales pueden evolucionar.

[1 Regions y Availability Zones](#)

[2 Modelo de responsabilidad compartida](#)

[3 Conceptos y capacidades de Amazon S3](#)

[4 Nombres y namespaces de buckets](#)

[5 Referencia de aws s3 sync](#)

[6 Crear una distribución de CloudFront](#)

[7 Acceso privado a S3 mediante OAC](#)

[8 Configuración de cache behaviors](#)

[9 Invalidación y versiones de archivos](#)

[10 Nombres de dominio alternativos](#)

[11 Precios y planes de CloudFront](#)

[12 Gestión del gasto con AWS Budgets](#)

[13 Certificados TLS para CloudFront](#)

[14 Qué es el cloud y sus modelos de servicio](#)

[15 Qué es Amazon Web Services](#)

[IR A LA CLASE 2](#)

